CS 559 - Machine Learning

STEVENS
INSTITUTE OF TECHNOLOGY
1870

Predicting NYC Citi Bike Station Congestion

Team Members

- Milan Chandiramani
- Komal Wavhal
- Matthew Schmitt

12 Dec 2025

Team Members

Milan G Chandiramani

CWID: 20032010

Komal Wavhal

CWID: 20034443

Matthew Schmitt

CWID: 20034443

Agenda

1. Introduction & Problem Statement
2. Challenges Facing CitiBike
3. Dataset Review, Data Cleaning & Pre-processing
4. Exploratory Data Analysis
5. Model Overview
6. Comparison & Conclusion
7. Future Work
8. References
9. Q&As

Introduction & Problem Statement

Citi Bike works best when bikes and docks are available at the right place and time. Congestion breaks this promise and creates frustration for both users and operators.

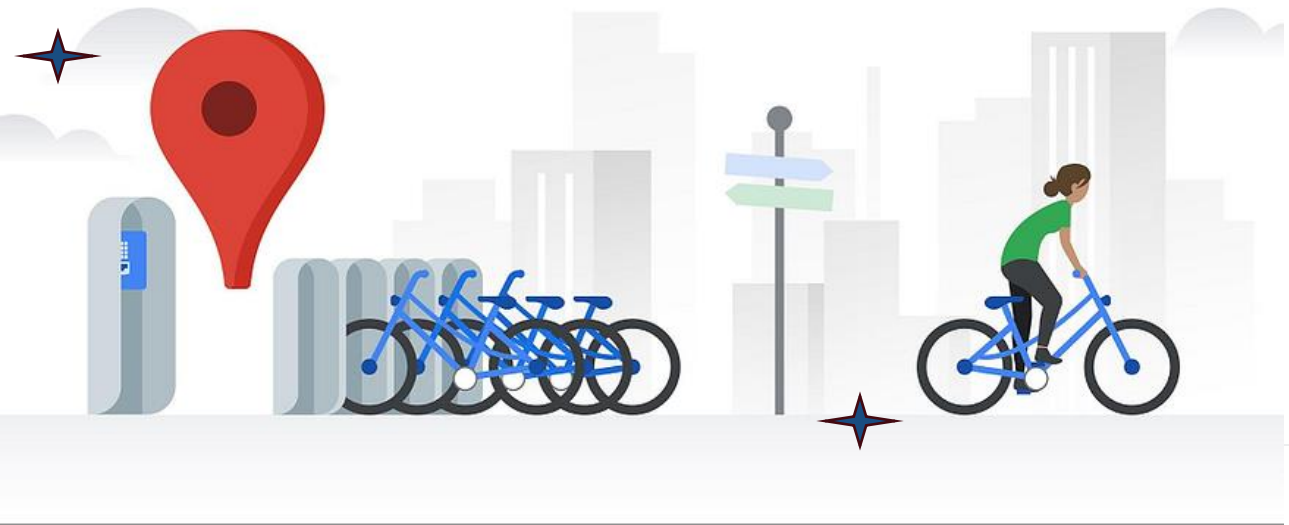
NYC Citi Bike is one of the largest bike-sharing systems in the world. Millions of trips daily → critical for urban mobility & sustainability

A recurring operational issue: station congestion

- ✓ Empty stations → no bikes
- ✓ Full stations → no docks

These failures directly impact:

- Rider satisfaction
- Operational cost
- System reliability



By combining high-resolution station data with modern machine learning, we show that urban bike congestion can be predicted accurately, at scale, and in real time

- **Dataset used:**
  - CitiBikeDataMyBusProcessed_occupancyfix.csv

Introduction & Problem Statement

Problem Statement: There is no reliable short-term forecasting system that predicts station congestion before it happens.

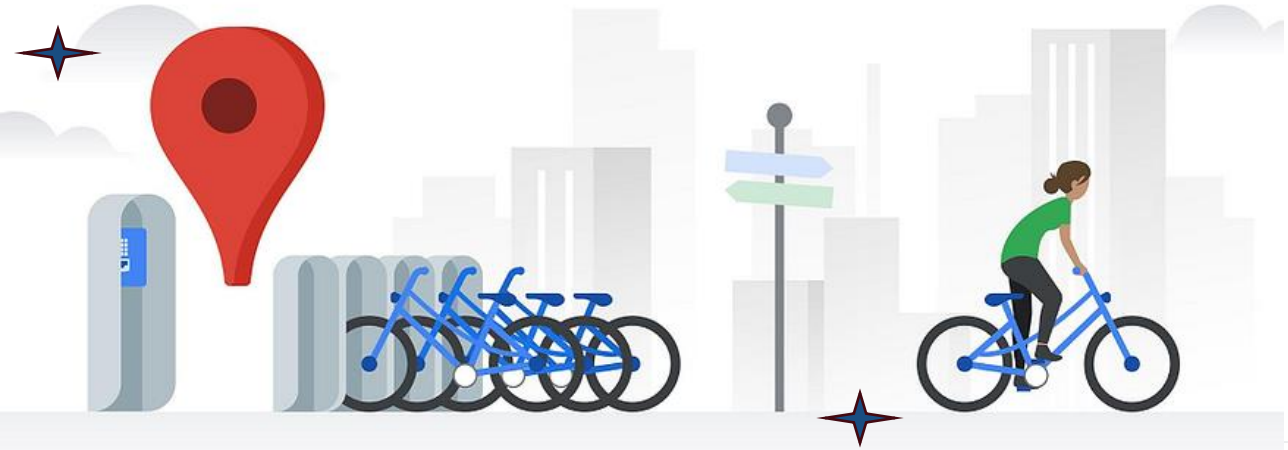
Most public Citi Bike datasets focus on:

- Trip start & end events
- Ride demand

Missing information:

- Real-time station availability
- Station occupancy dynamics

Operators currently react after congestion occurs



Goal of The Project

Forecast station-level operational conditions at an hourly resolution
Predict:

- ✓ Available bikes
- ✓ Available docks
- ✓ Station occupancy ratio

Compare classical time-series models vs modern ML models
Deploy results in a real, interactive system

Success Criteria:

- ✓ High predictive accuracy
- ✓ Scalability across all stations
- ✓ Real-time usability via deployment

Motivation

For users

- Fewer failed trips
- Better planning

For operators

- Proactive bike rebalancing
- Reduced operational costs

For cities

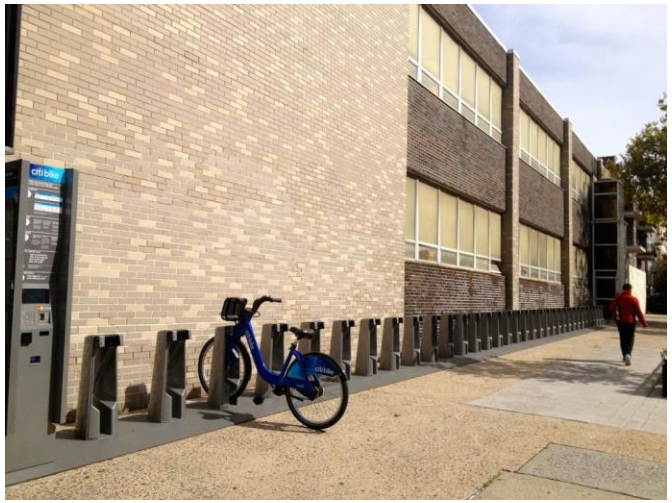
- Improved sustainability
- Smarter transportation planning

Key Insight: Predicting congestion before it happens is more valuable than reacting after failure.

Challenges

- Many neighborhoods in the city do not have racks/stations in the most convenient of areas.
- Customers occasionally experience issues with receiving broken equipment or e-bikes that are not adequately charged.

- Stations, particularly during peak commuting hours, either do not have enough bikes to satisfy customer demand or are full and unable to accept additional bikes that customers are attempting to drop-off - this will be the area of concern regarding our project.



Challenges

Data challenges

- Large scale (23+ million records)
- Missing & noisy station signals

Modeling challenges

- Strong daily & weekly seasonality
- Abrupt congestion spikes

System challenges

- Scalability across hundreds of stations
- Need for real-time inference

Why this is hard: Simple linear models fail to capture nonlinear demand, while deep models risk overfitting and instability.

Data Collection

Dataset Screenshot

	dock_name	_lat	_long	avail_bikes	avail_docks	tot_docks	occupancy	is_holiday	timestamp	is_weekend	temperature_2m	relative_humidity_2m	precipitation	wind_speed_10m
0	"W 52 St & 11 Ave"	40.767272	-73.993929	7.0	32	39.0	0.179487	0	2016-02-01 14:09:00	0	10.6	56.075845	0.0	2.6
1	"W 52 St & 11 Ave"	40.767272	-73.993929	6.0	33	39.0	0.153846	0	2016-02-01 14:31:00	0	10.6	56.075845	0.0	2.6
2	"W 52 St & 11 Ave"	40.767272	-73.993929	4.0	35	39.0	0.102564	0	2016-02-01 21:04:00	0	11.7	48.186541	0.3	3.1
3	"W 52 St & 11 Ave"	40.767272	-73.993929	5.0	34	39.0	0.128205	0	2016-02-01 21:18:00	0	11.7	48.186541	0.3	3.1

Column names:

- dock_name
- _lat
- _long
- avail_bikes
- avail_docks
- tot_docks
- occupancy
- is_holiday
- timestamp
- is_weekend
- temperature_2m
- relative_humidity_2m
- precipitation
- wind_speed_10m

Data Collection

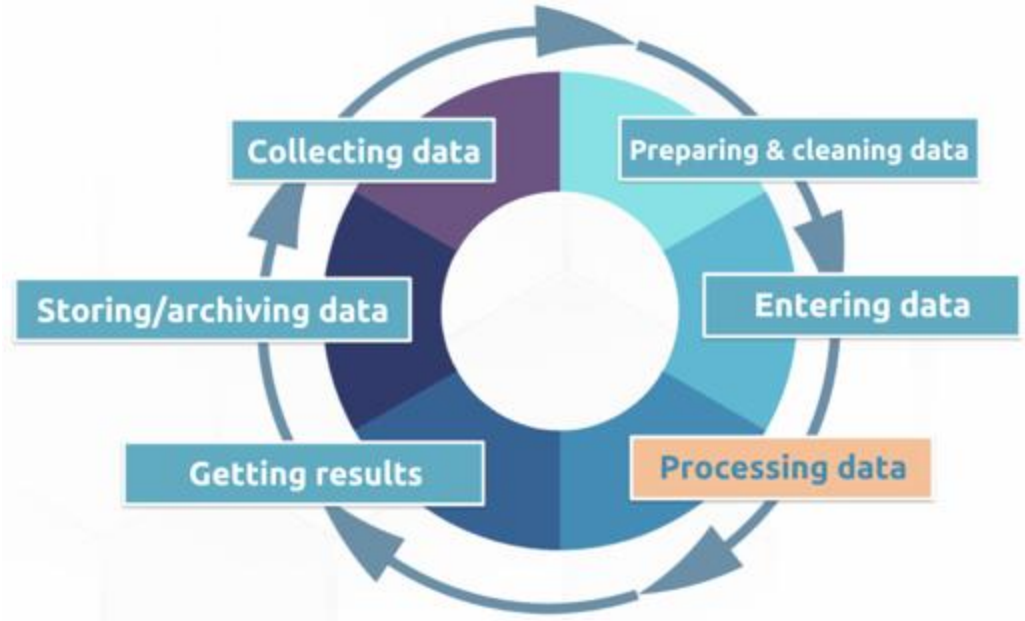
The efficacy of our predictive modeling relies on the selection of high-granularity, multi-dimensional datasets that capture the dynamic nature of urban mobility. The longitudinal scope of this data spans from **March 2015 to April 2019**, providing a significant historical window for detecting long-term trends and seasonal patterns.

1.1 Core Transactional Data

Our primary objective was to source a dataset that captured the micro-level mechanics of bike-sharing systems. We prioritized event-based logs where every individual station transaction—specifically the insertion or removal of a bike—was recorded as a discrete event.

The selected dataset, sourced from **The Open Bus**, provided four critical dimensions for each entry:

Station Identity: Descriptive station names stored as strings.
Temporal Precision: High-fidelity timestamps accurate to the nearest second.
Inventory Status: Real-time counts of bikes currently present at the station.
System Capacity: The total dock capacity available at each location.



Data Collection

Environmental Augmentation

- Through iterative testing, it was determined that internal station metrics alone were insufficient for high-accuracy demand forecasting. User behavior in bike-sharing systems is inextricably linked to environmental variables.
- To account for these external drivers, we integrated a secondary dataset from the National Weather Service's Integrated Surface Database

Integrated Surface Database (NWS-ISD)

- This provided a localized snapshot of weather conditions synchronized with our core transactional data. By pairing system demand with meteorological telemetry, we created a comprehensive feature set capable of capturing the impact of weather-induced fluctuations on ridership.

Data Processing

To ensure the highest level of analytical rigor, the raw dataset underwent a multi-stage preprocessing pipeline designed to eliminate noise, enforce schema integrity, and enrich the core features.

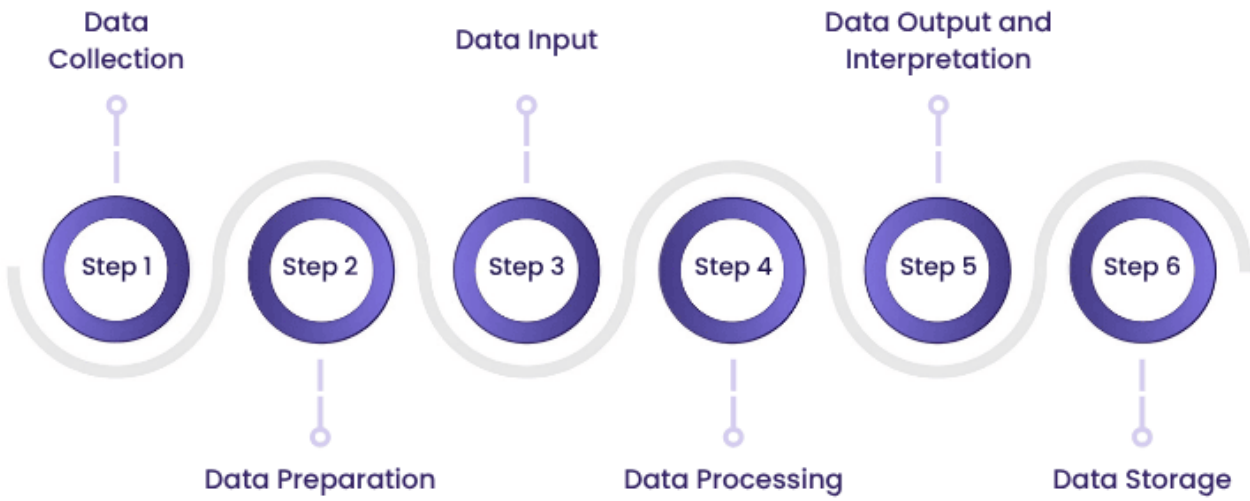
Data Sanitization and Validation

Our initial focus was on establishing a "ground truth" for the location data. We applied strict validation rules to the primary identifiers:

Schema Enforcement: We filtered for dock_name and dock_id, ensuring the former remained strictly alphanumeric and the latter purely numeric to prevent data corruption.

Business Logic Filtering: Stations prefixed with "coming soon" were programmatically
removed to ensure the analysis reflects only operational, real-world infrastructure.

Dimensionality Reduction: We performed a thorough "junk data" audit, pruning
non-pertinent features (e.g., MMT, _long3) to optimize computational efficiency and model focus.



Dataset Review

Temporal Reconstruction and Feature Engineering

To move from raw observations to time-series insights, we transformed discrete variables into high-fidelity temporal markers:

Timestamp Synthesis: Using Pandas, we vectorized the date, hour, and minute columns to generate unified **UTC timestamps**, enabling precise synchronization with external datasets.

Temporal Categorization: We engineered new Boolean features to flag **weekends** and **federal holidays**, accounting for the significant shifts in user behavior typically seen during non-working days.

Occupancy Metric: An intermediate piece of data/feature was provided that allows the models to view the occupancy ratio of any given station during an event.

Environmental Contextualization (NWS Integration)

Recognizing that bike-sharing demand is heavily contingent on environmental factors, we augmented our dataset with high-resolution weather telemetry:

Geospatial Join: Utilizing the NWS-ISD (Integrated Surface Database), we retrieved localized weather data by cross-referencing the latitude, longitude, and synthesized timestamps of each event.

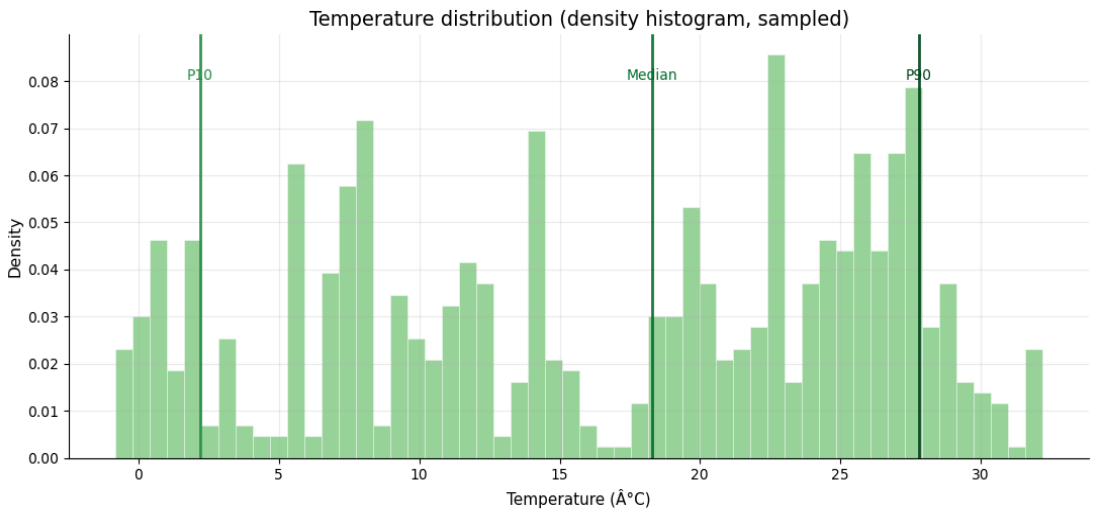
Feature Enrichment: After executing necessary unit conversions, we integrated four critical environmental vectors **Precipitation, Temperature, Relative Humidity, and Wind Speed**—directly into the primary data-frame.

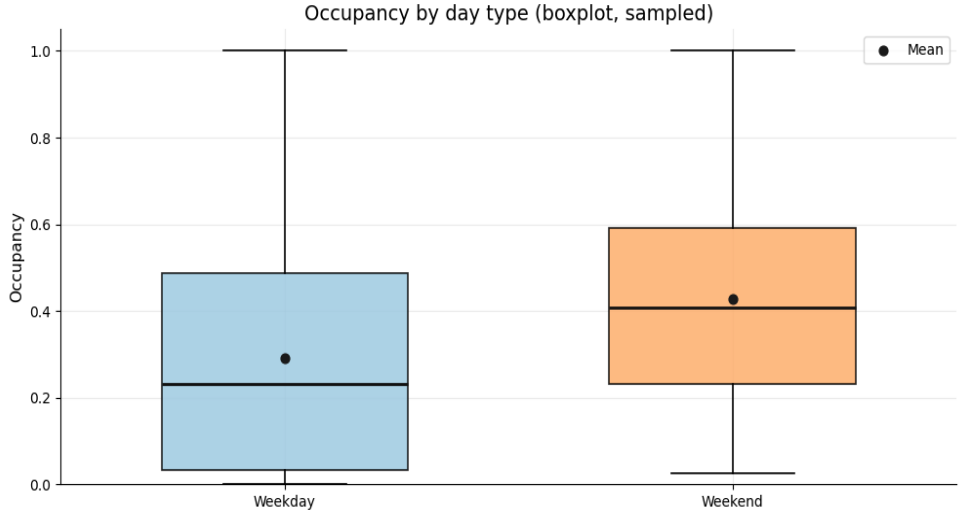
Data Processing Statistics

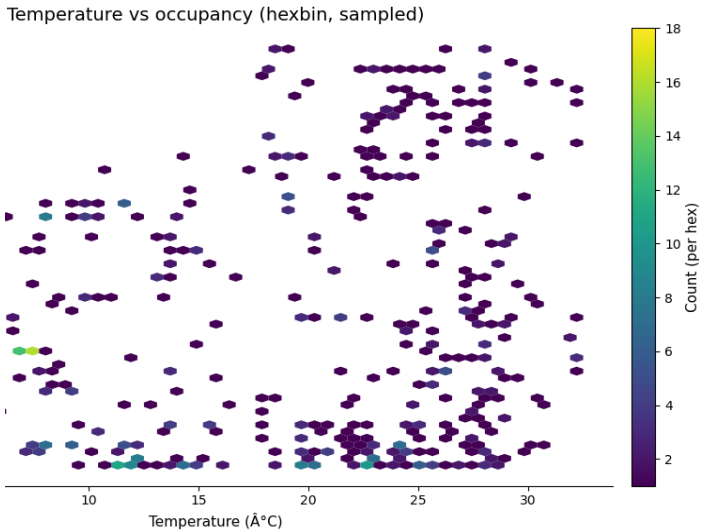
- Hourly dataset size: 2392762 rows
- Train shape: (632483, 16), Test shape: (1760279, 16)

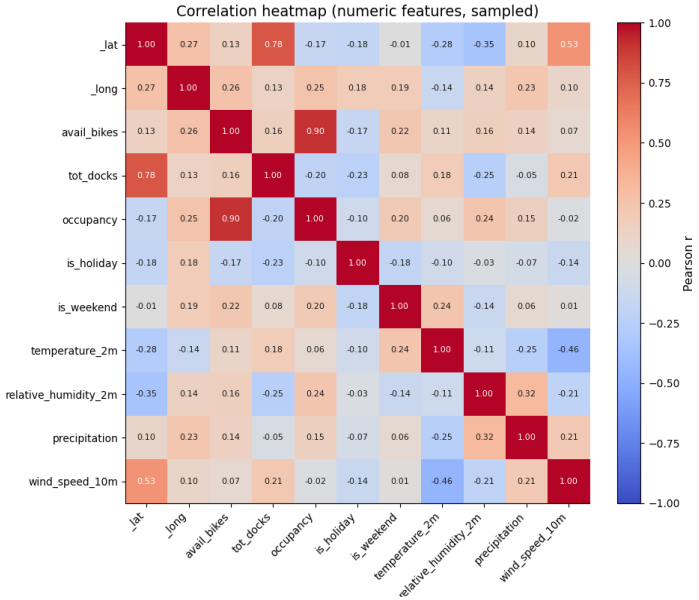
- **Missing Values:** Within the features that we are concerned with, we observe 10 missing values in our dataset.
- **Duplicates:** No duplicates were observed.
- **Outliers:** The outliers in our data primarily constitute demand during severe weather events, such as winter storm Toby in 2018
- **Data Type:** Our final data frame consisted of Pandas timestamps, integers, 64-bit precision floats, and strings. No Categorical data types present.
- **Feature Scaling:** Standard scalar

Data Processing Statistics

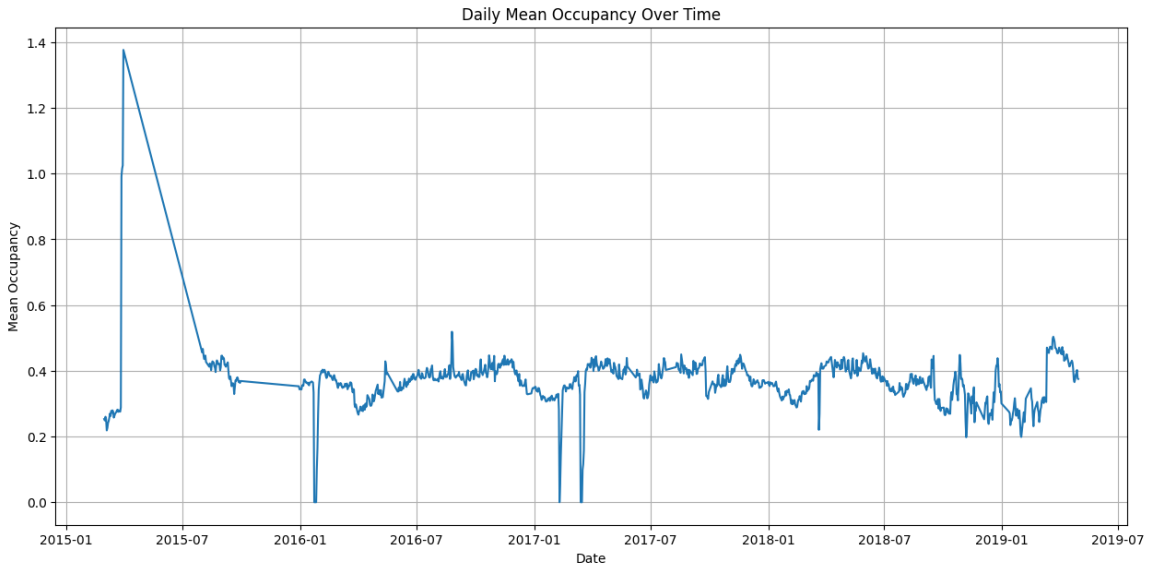


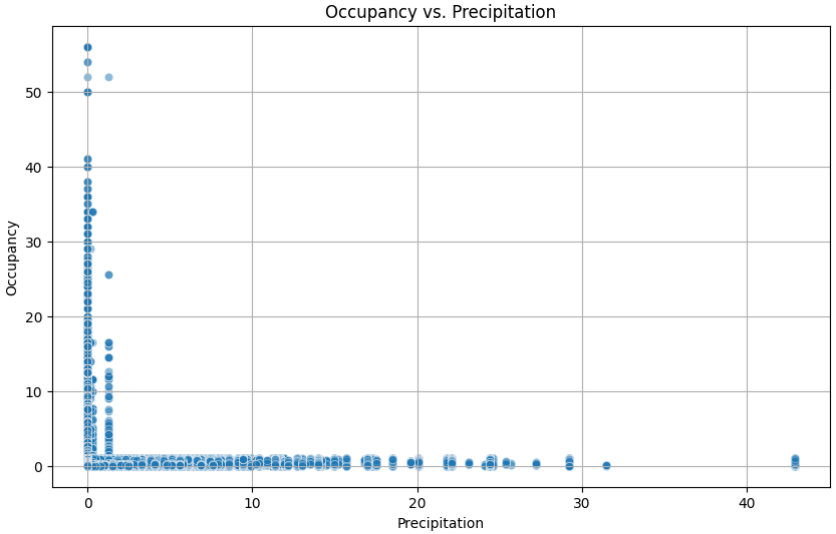


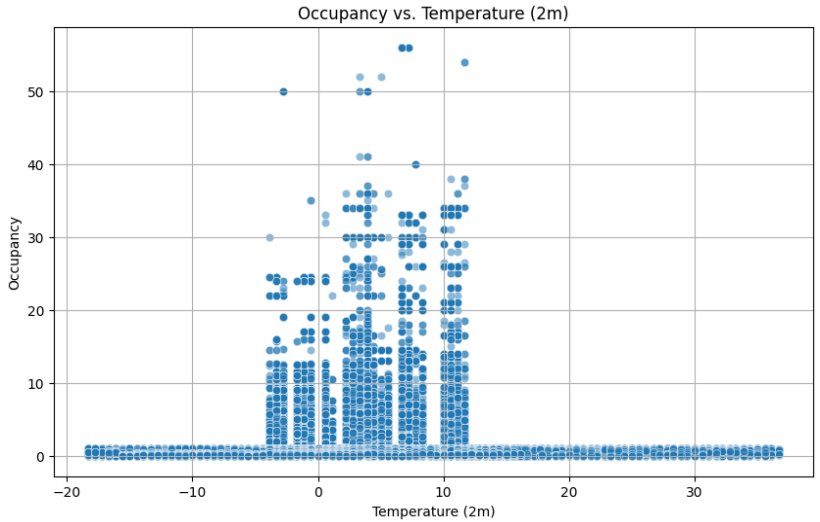


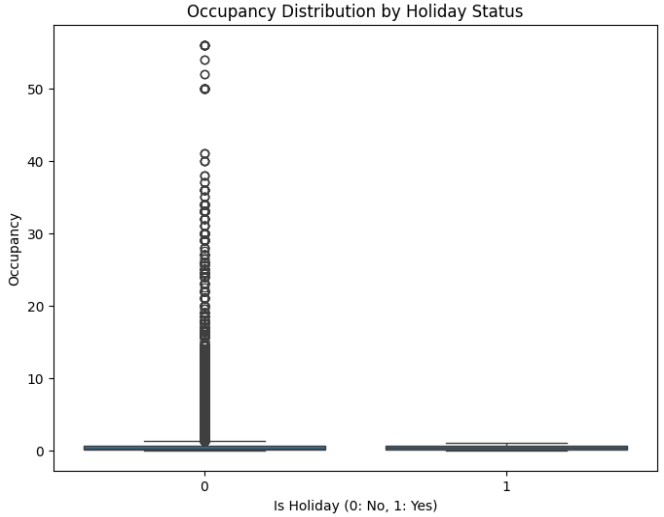


Descriptive Charts for Dataset









Project Process Flow

Problem Definition

API

CSV File
(The open Bus)

Data Load

Data Cleaning & Pre-processing

Exploratory Data Analysis

Model Selections
ARIMA/XGBoost/
LSTM/Prophet

Model Prediction

Model Evaluation and Model Saving into .pkl/keras/h5

Future Scope

Realtime Forecasting

Streamlit Application

Streamlit

Model Overview – ARIMA

What is ARIMA?

- A classical statistical time-series model **(Statistical Baseline)**
- Used for forecasting values using past observations + past errors
- Assumes data can be made stationary (constant mean/variance over time)

ARIMA Model Form

$$y_t = c + \sum_{i=1}^{p} \phi_i \, y_{t-i} + \sum_{j=1}^{q} \theta_j \, e_{t-j} + e_t$$

ARIMA stands for:

A — Autoregressive (AR, p)

- Model uses past values of the series
- Example: predicting congestion using congestion in previous hours
- Equation term: $y_{t-1}, y_{t-2}, \ldots$

I — Integrated (I, d)

- Differencing applied to remove trends
- Makes time series stationary
- Example: $y'_t = y_t - y_{t-1}$

MA — Moving Average (MA, q)

- Model uses past forecast errors
- Example: shocks or deviations from expected values
- Equation term: $e_{t-1}, e_{t-2}, \ldots$

Model Overview – ARIMA

Why ARIMA for Citi Bike Congestion?

- Captures hour-to-hour demand trends
- Effective for short-term forecasting
- Good baseline to compare with ML models (RF, XGBoost, LSTM)

Strengths:

- Easy to interpret
- Captures short-term trends

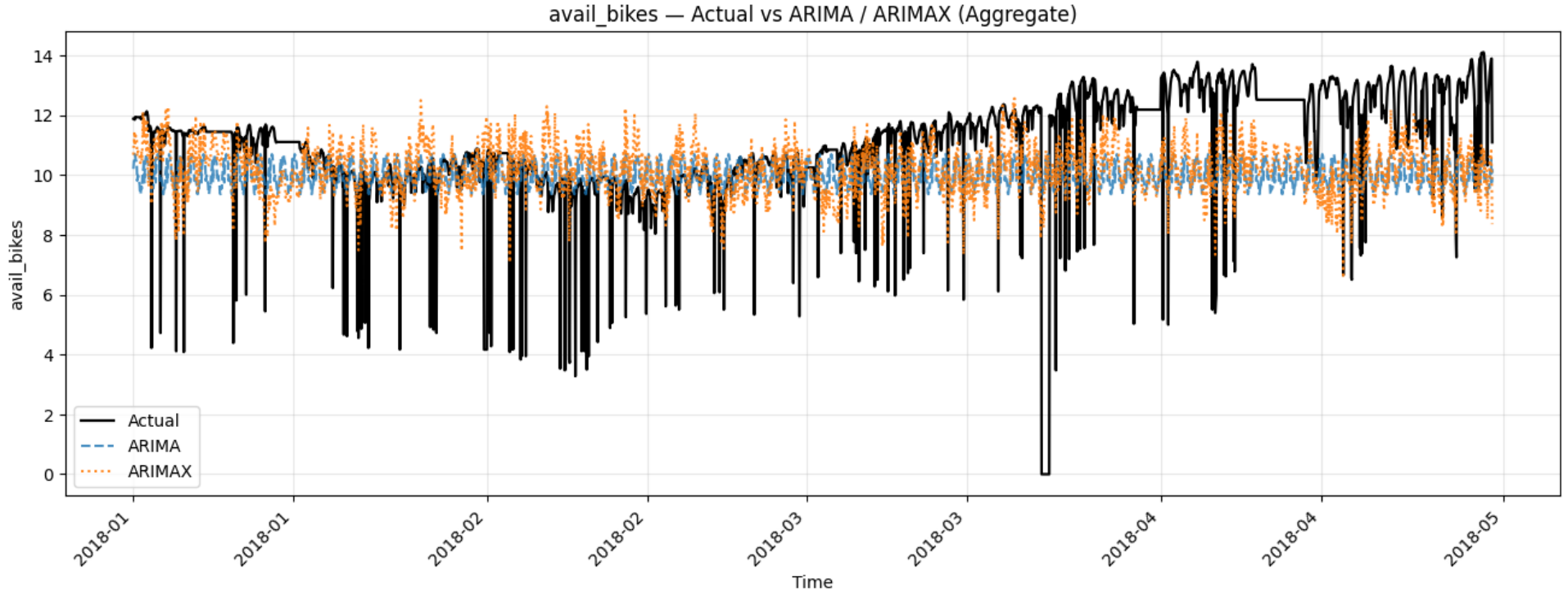
Limitations in Our Use-Case

- Requires station-level data to be stationary → differencing
- Assumes linearity
- Multi-station generalization
- Struggles with:
  - Nonlinear congestion patterns
  - Seasonality without SARIMA
  - External predictors (weather, holidays)
- Not ideal for spatio-temporal dependencies between stations

ARIMA provides a useful baseline but is not sufficient for complex urban systems.

Model Results – ARIMA

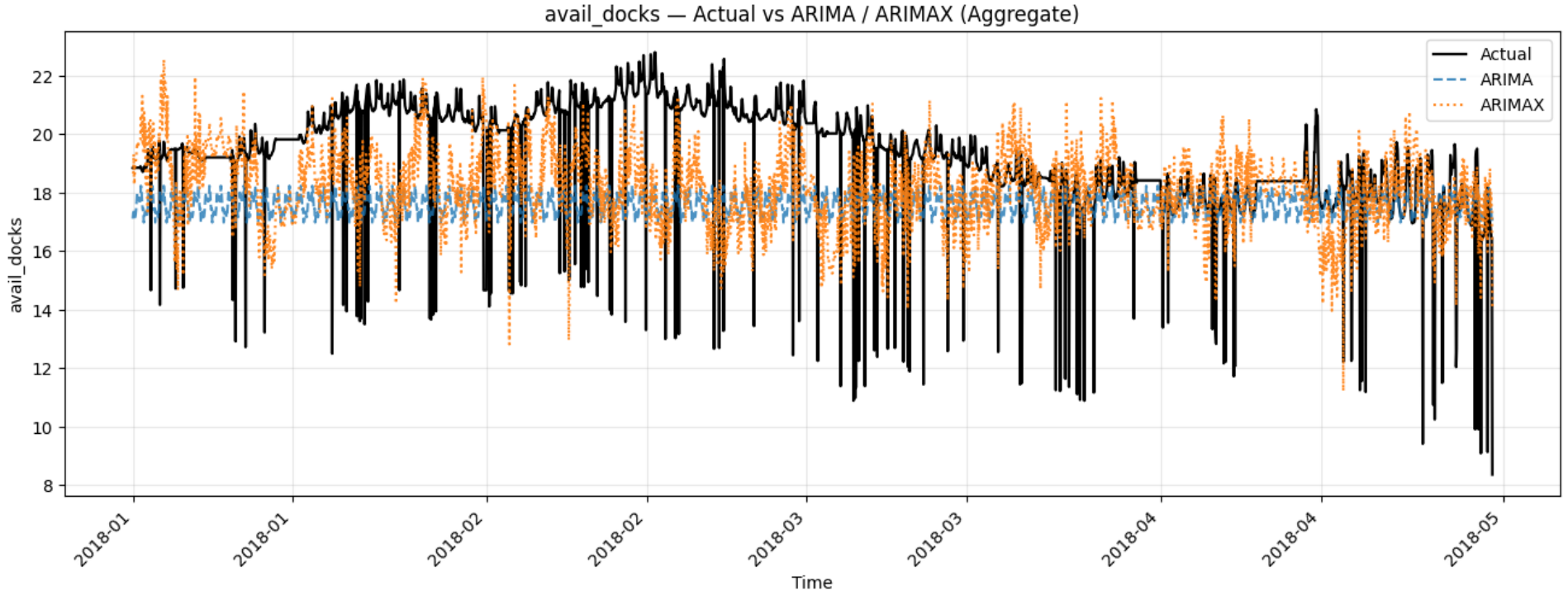
- ARIMA captures the **overall temporal trend** in bike availability but produces **smoothed predictions**.
- ARIMAX slightly improves short-term variability by incorporating **exogenous features** (e.g., time-based or contextual signals).
- Both models struggle to capture **sharp drops and sudden spikes**, which correspond to **rapid station depletion events**.
- Extreme dips in actual values highlight the **limitations of linear time-series models** under highly dynamic demand.
- These results motivate the use of **nonlinear ML models (XGBoost, LSTM)** for better peak and shock handling.



ARIMA works well for trend tracking but underestimates abrupt changes in bike demand.

Model Results – ARIMA

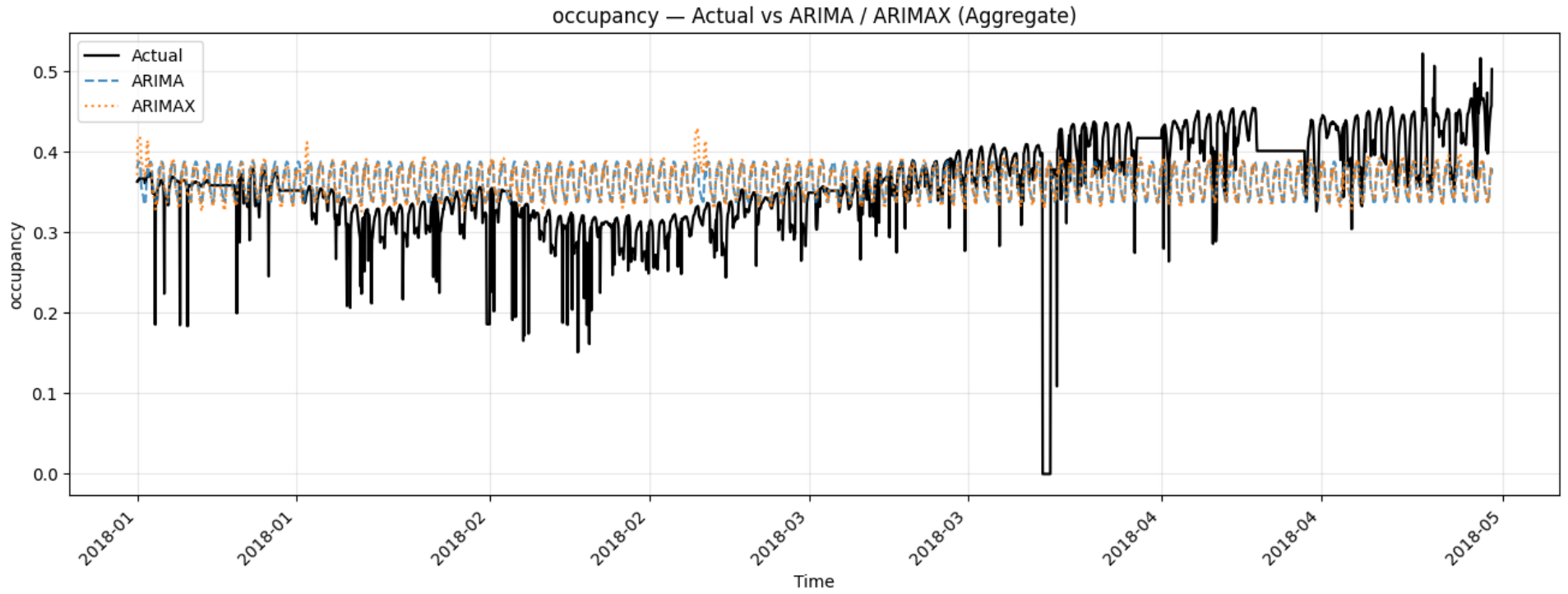
- Dock availability shows **strong inverse behavior relative to bike availability, validating data consistency**.
- ARIMA produces **stable but mean-biased predictions**, **failing to adapt to rapid dock** saturation.
- ARIMAX introduces higher variance and **responsiveness, improving alignment** during moderate fluctuations.
- Large deviations during peak periods indicate **capacity constraints** and **rebalancing** inefficiencies.
- Results demonstrate that dock availability is highly event-driven, limiting purely statistical forecasting approaches.



This highlights why dock congestion is harder to predict using classical linear models.

Model Results – ARIMA

- Occupancy forecasts **exhibit strong daily seasonality**, which ARIMA captures reasonably well.
- Both ARIMA and ARIMAX **produce over-smoothed occupancy curves**, missing sudden **occupancy collapses**.
- ARIMAX provides marginal improvement by leveraging **exogenous signals** but still lacks reactivity to shocks.
- Sharp occupancy drops correspond to full/empty station states, critical for congestion risk detection.
- This motivates framing congestion as a classification problem (full/empty risk) rather than pure regression.



Occupancy smoothness hides congestion risk—classification can better capture operational failures.

Model Results – ARIMA / ARIMAX

❖ Key Takeaways

1. Effective for baseline trend and seasonality modeling
2. Limited ability to handle nonlinear demand spikes
3. ARIMAX improves stability but not extreme behavior
4. Best suited as benchmark or interpretability baseline
5. Reinforces the need for ML and deep learning models for operational forecasting

Table 3: ARIMA and ARIMAX Forecasting Performance Across All Targets

Model	Target	RMSE	MAE	R^2	MAPE (%)
ARIMA	Available Bikes	2.62	2.19	0.012	25.53
ARIMAX	Available Bikes	2.57	2.08	0.046	24.58
ARIMA	Available Docks	2.52	1.96	0.018	11.87
ARIMAX	Available Docks	2.68	2.13	-0.116	12.22
ARIMA	Occupancy	0.066	0.052	0.101	15.94
ARIMAX	Occupancy	0.066	0.052	0.115	15.89

Model Overview – XGBoost

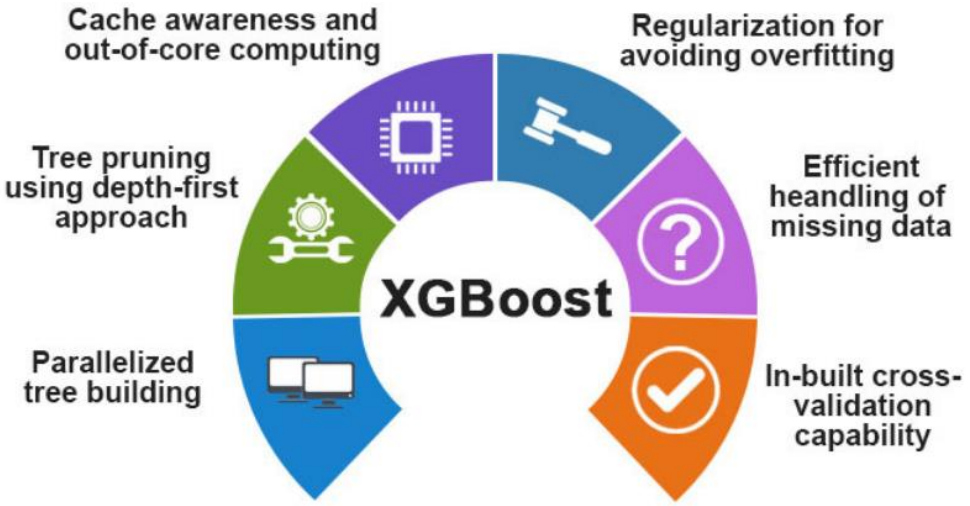
What is XGBoost?

A gradient-boosted decision-tree model **(Primary Forecasting Model)**
Optimized for speed, accuracy, and large datasets
Works extremely well with nonlinear patterns in congestion data
Learns from engineered features (lags, weather, time)

How XGBoost Works

- Combines many weak decision trees into a powerful model
- Each tree corrects errors of the previous trees
- Uses gradient boosting to minimize prediction error

$$\hat{y}_t = \sum_{k=1}^{K} f_k(X_t), f_k \in \textit{decision trees}$$



XGBoost Algorithm

Limitations

- Does not model sequential patterns as deeply as LSTMs
- Requires careful feature engineering
- Forecast horizon limited (must recursively predict for multi-step forecasts)

Model Overview – XGBoost

Feature Set for Citi Bike Congestion:

1. **XGBoost can use expanded feature engineering, e.g.:**
2. **Temporal Features**
   - Hour of day (sin/cos)
   - Day of week
   - Month, holiday
3. **Lagged Congestion Features**
   - Past 1–24 hour arrival counts
   - Rolling means (3 hr, 6 hr, 24 hr)
4. **Weather Features** - Temperature, humidity, rain, wind
5. **Station Metadata**
   - Dock capacity
   - Station category (Hoboken Terminal, Path/Transit, Residential)

Why XGBoost Works Best:

- No stationarity assumption
- Learns complex feature interactions
- Robust to noise and sudden demand shifts

Results Highlight (important slide callout):

✓ Highest R^2 across all targets
✓ Lowest RMSE & MAE
✓ Most stable predictions during peak congestion

Why XGBoost for Congestion?

- Captures complex, nonlinear relationships between features
- Handles weather + demand interactions well
- Requires no stationarity
- Very strong predictive performance on structured data

Handles:

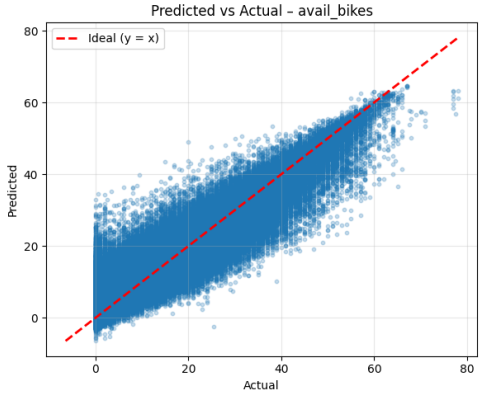
- Nonlinear interactions
- High-dimensional features
- Large datasets efficiently

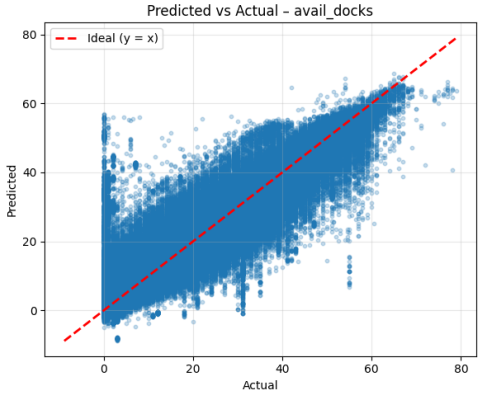
Global multi-output regression:

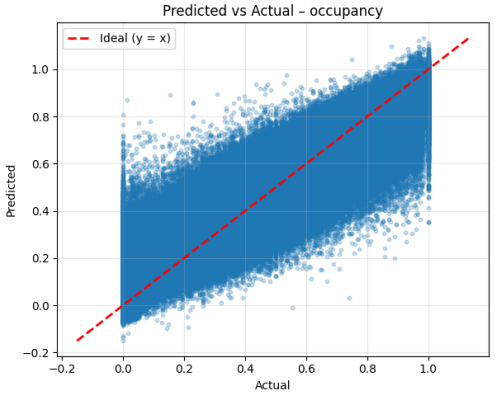
- Bikes
- Docks
- Occupancy

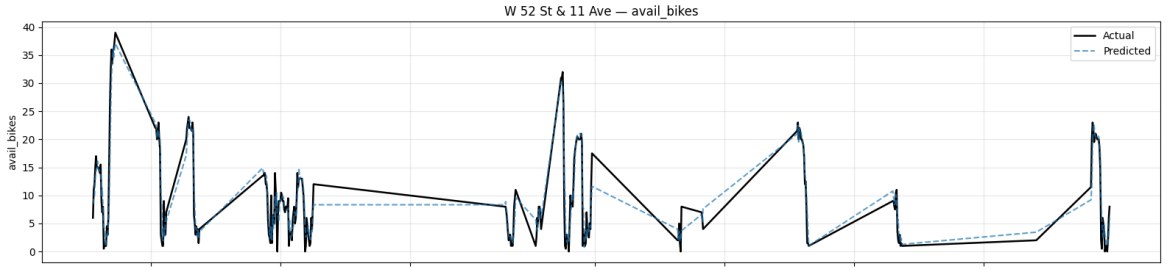
XGBoost consistently outperformed both ARIMA and LSTM.

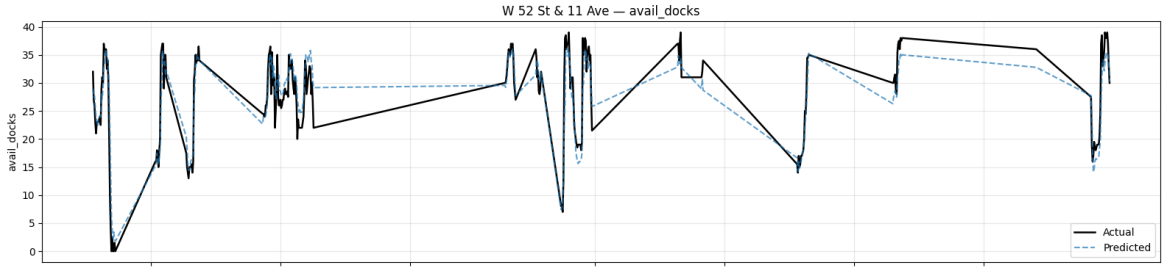
Model Result – XGBoost

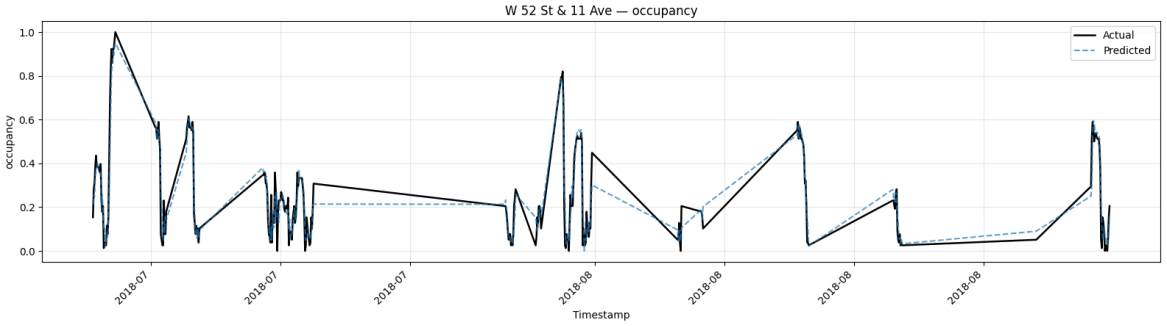


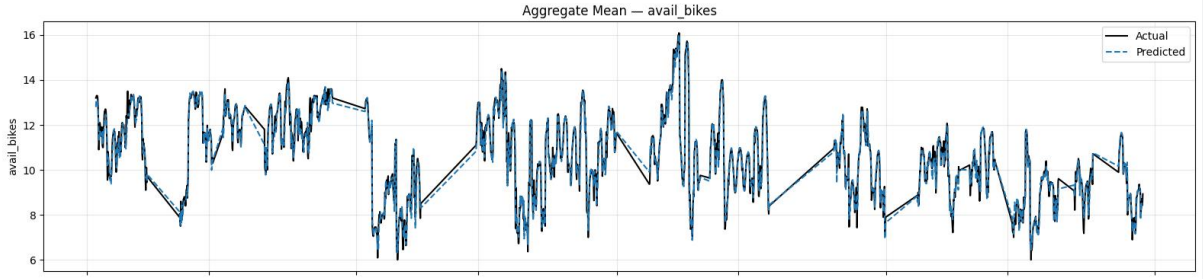


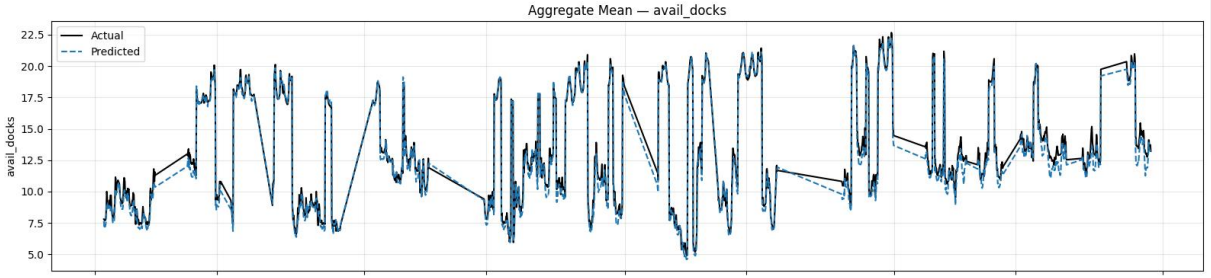


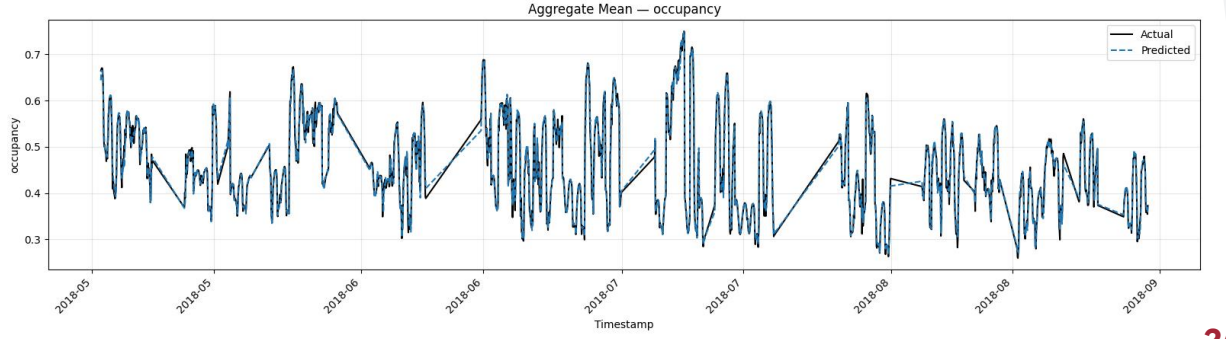








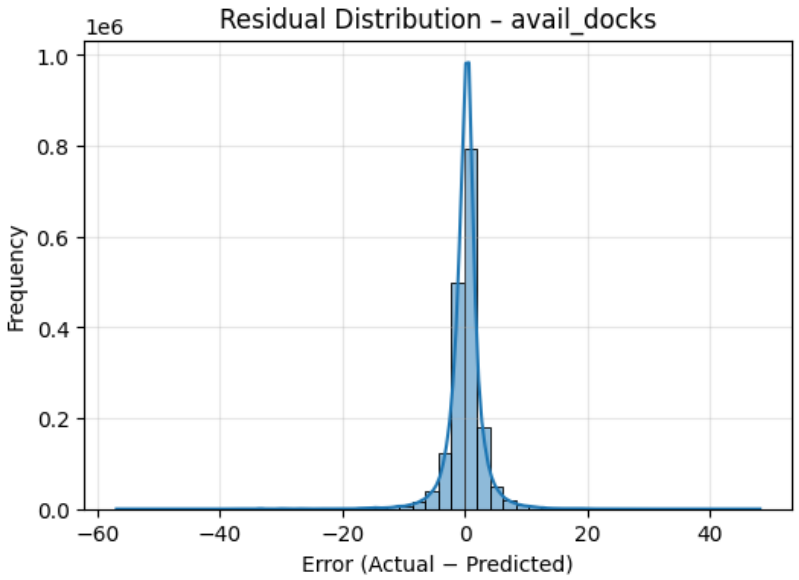


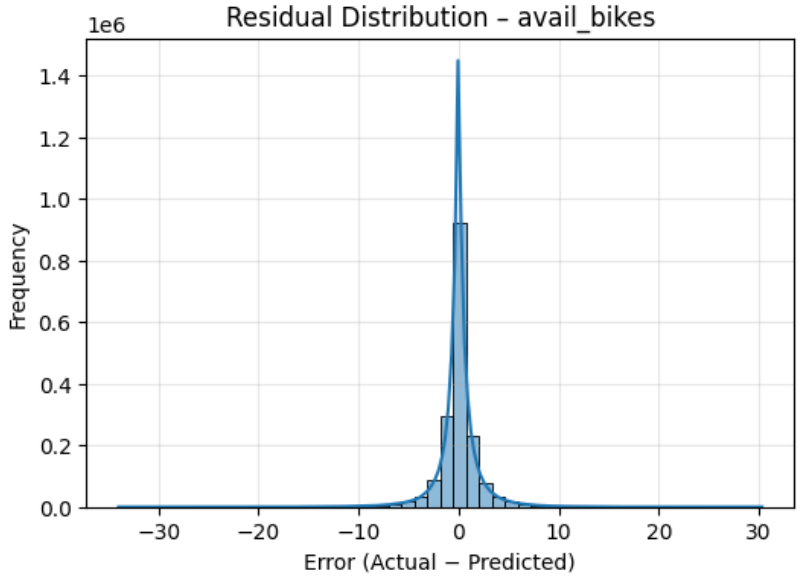


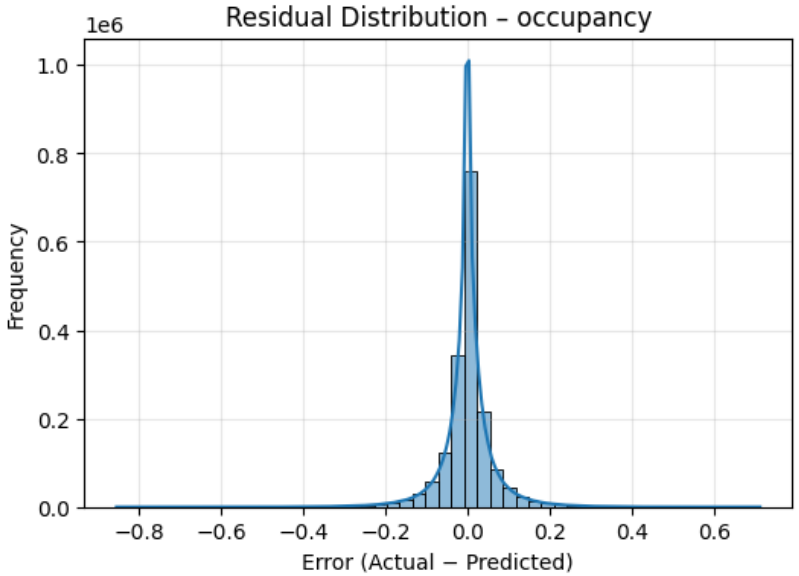
Model Result – XGBoost

Table 2: XGBoost Forecasting Performance Across All Targets

Target	RMSE	MAE	R^2	MAPE (%)
Available Bikes	2.09	1.18	0.957	23.19
Available Docks	3.09	1.81	0.940	22.46
Occupancy	0.061	0.037	0.959	22.76







Model Overview – LSTM

- Global multi-station Sequence-to-one global LSTM
- Uses 48-hour historical windows
- Target forecasting outputs : available bikes, available docks, occupancy
- Captures long-range non-linear and temporal dependencies

Why LSTM:

- No stationarity assumption
- Learns complex feature interactions
- Robust to noise and sudden demand shifts
- Idle for Spatio Temporal , Multi-Variate Time Series Data

Limitations:

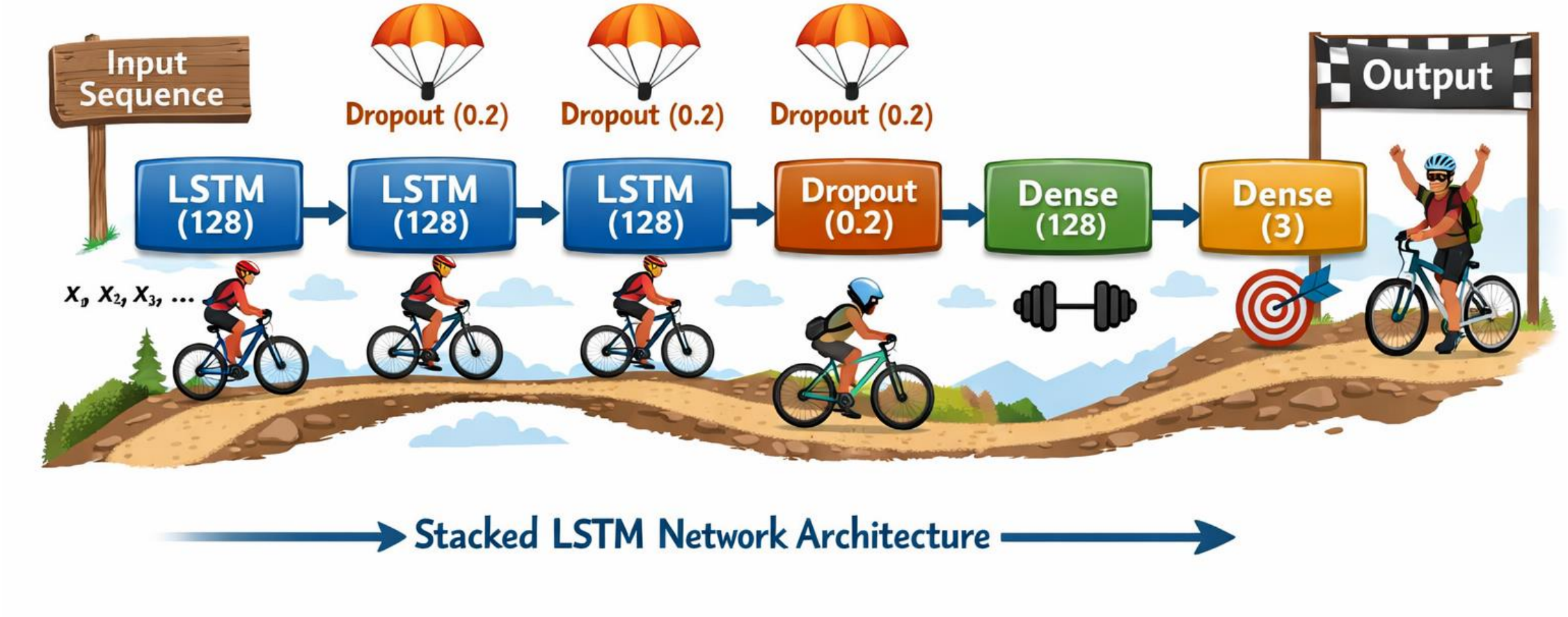
- ✓ Overkill complex model takes longer time to train

Observation:

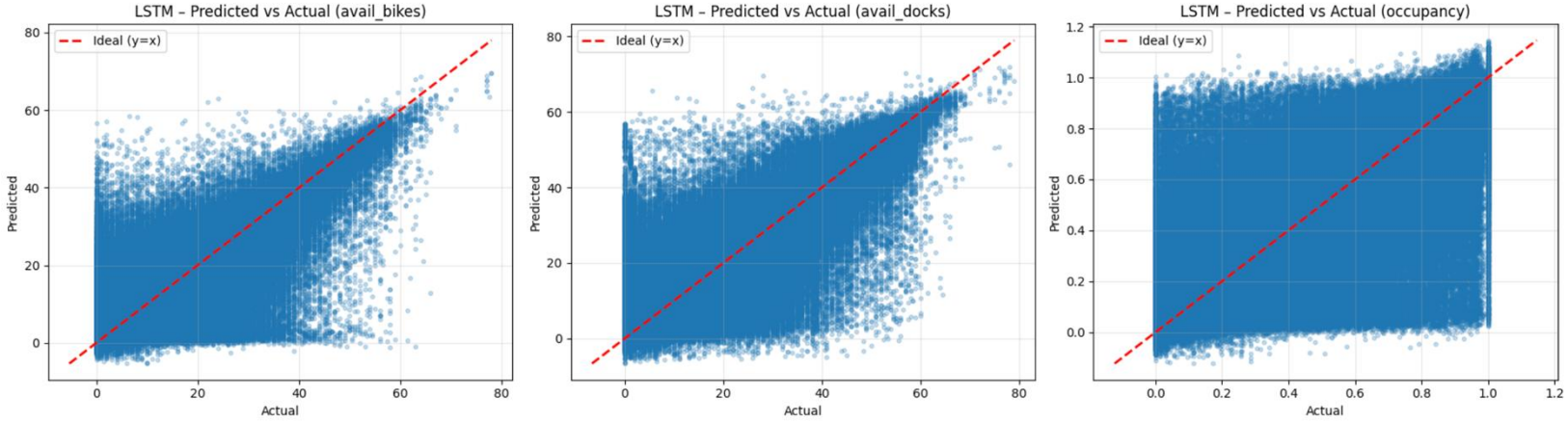
- Smooth forecasts
- Struggles with abrupt congestion spikes

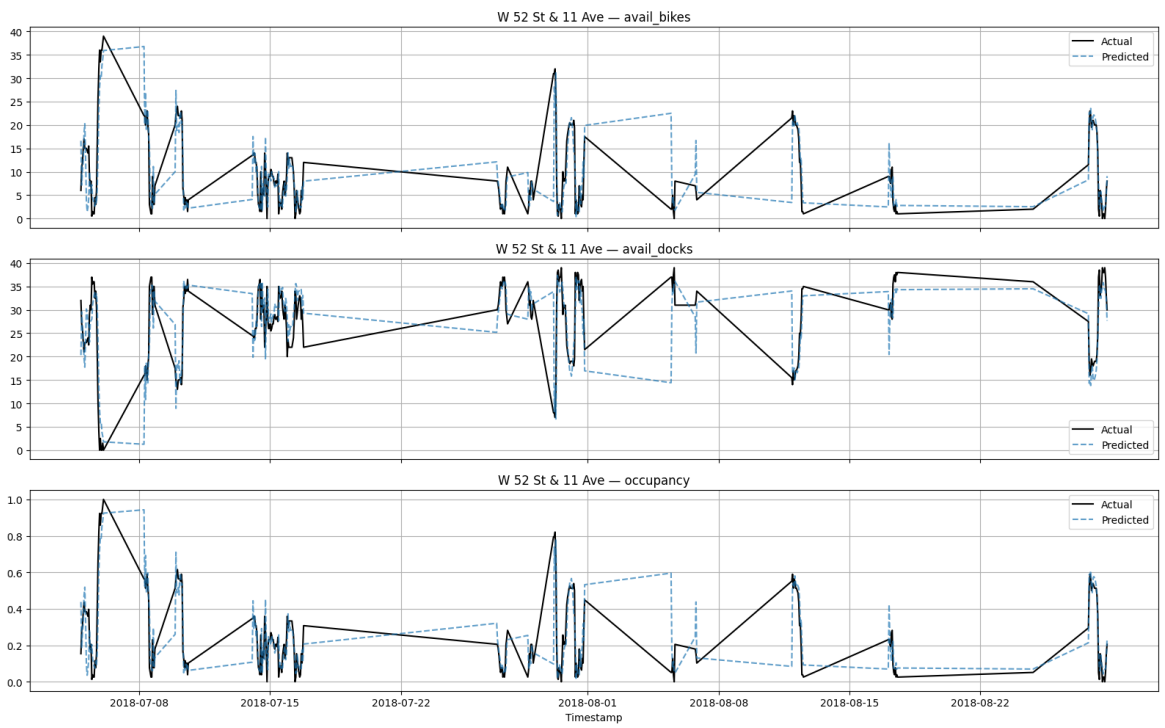
Conclusion: LSTMs learn trends well but are less responsive than tree-based models for operational forecasting.

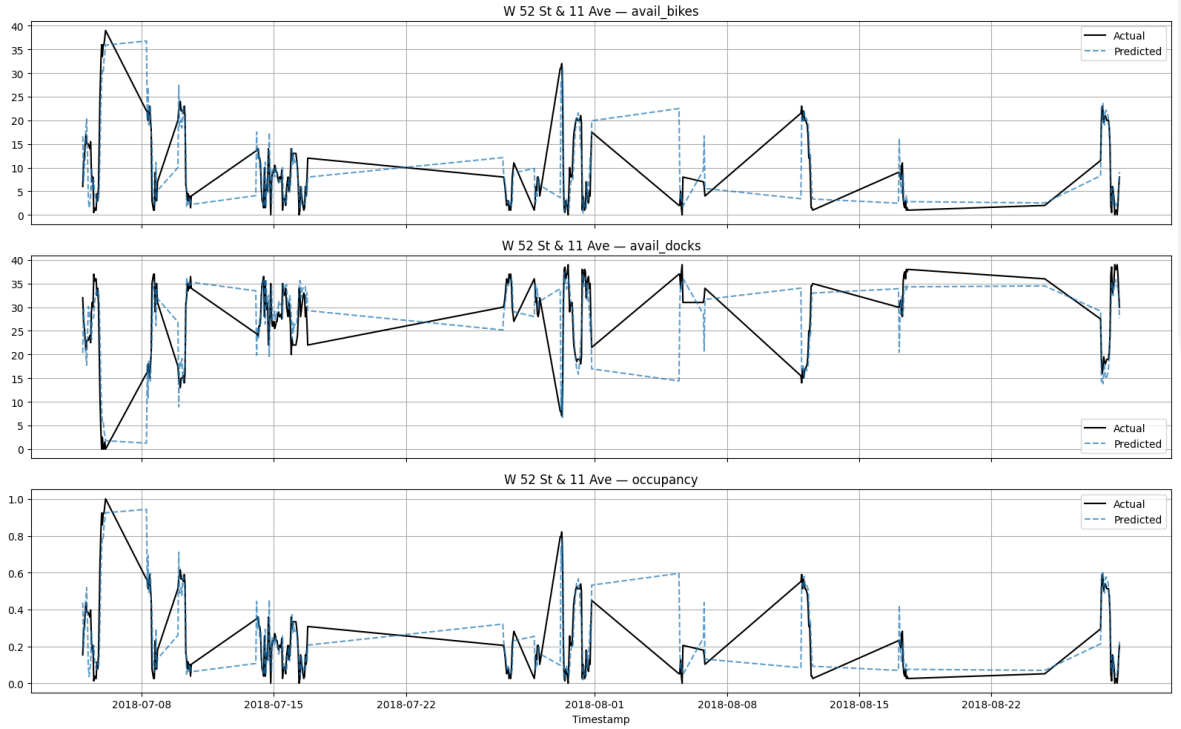
Model Overview – LSTM



Model Result – LSTM



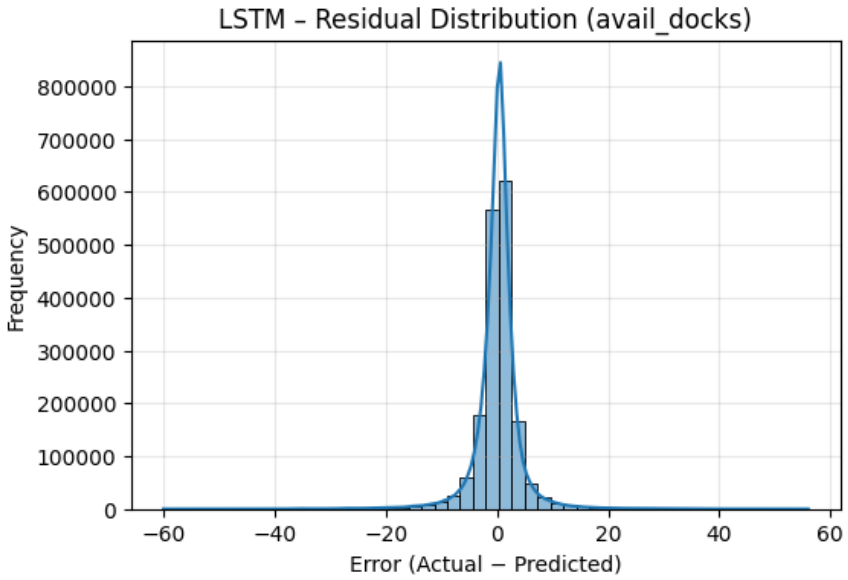


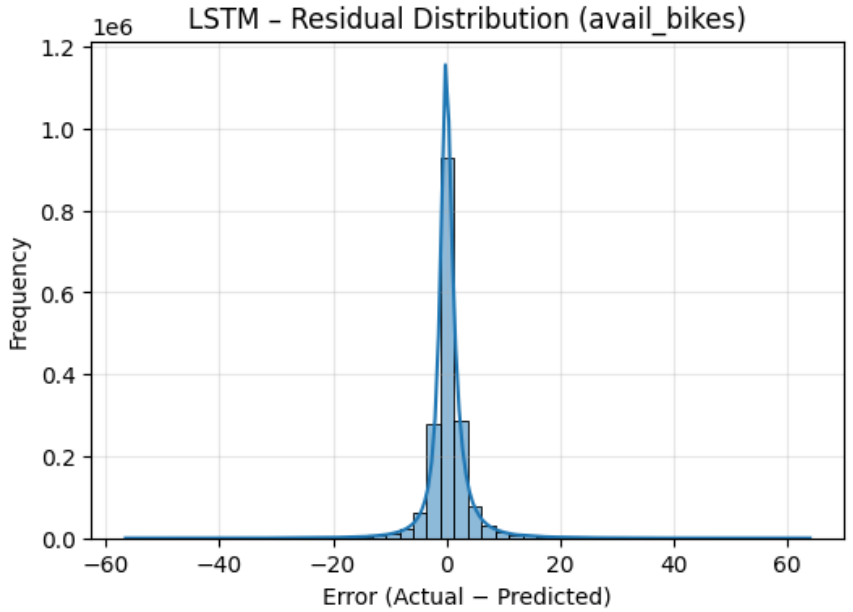


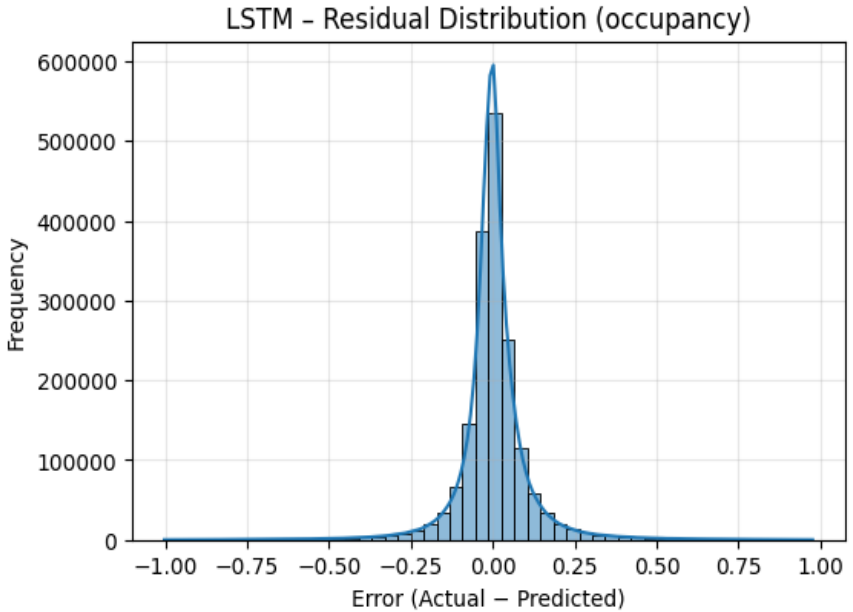
Model Result – LSTM

Table 1: LSTM Forecasting Performance Across All Targets

Target	RMSE	MAE	R^2	MAPE (%)
Available Bikes	3.75	2.10	0.862	39.13
Available Docks	4.31	2.49	0.884	31.90
Occupancy	0.111	0.065	0.867	38.94

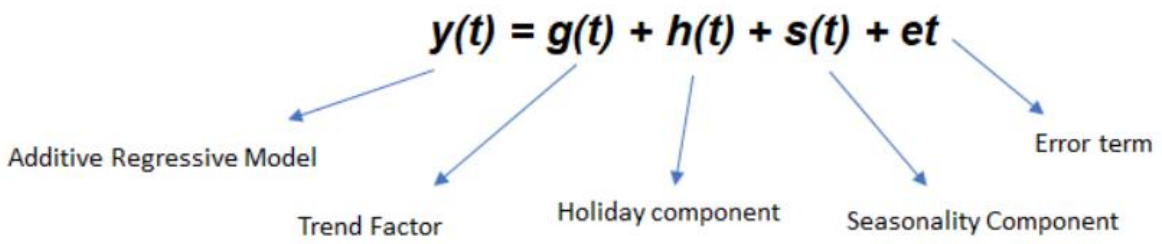






Model Overview – Prophet

- Prophet is an additive and multiplicative regression model that decomposes trends into multiple pieces: the main trend, seasonality, holidays, and outlier events/noise.
- These individual pieces are represented as linear or logarithmic regressions (default is piecewise linear).
- Prophet is most effective on datasets that have some form of linear or logarithmic trend with well-defined exceptions.
- Prophet is very sensitive to variance that is not foretold to it when setting parameters.
- It is designed to predict strictly one target feature.



Prophet Forecasting Model Equation

Model Overview –Prophet

- **g(t) - trend segment**
  - The user has the choice whether to use linear or logistic models
  - Implements a regularization that is essentially what we know as L1 regularization
  - The user can manually elect to insert turning points in the graph if that would assist the model
- **s(t) - seasonal segment**
  - Accounts for seasonal trends that occur with a period defined by the user (ex: weekly, yearly, or daily)
  - These trends are accounted for with Fourier series
  - Seasonal trends can provide additive or multiplicative feedback to the main trend
- **t(t) - holiday segment**
  - Set windows capture abnormalities relating to holidays or other special events
- **et – error/noise term**
- **The prophet model also provides easy methods to allow for cross-validation and the production of statistics regarding model performance**

Model Result – Prophet

- **Our prophet model was trained on our dataset with the intent of predicting occupancy values for the CitiBike system at any given time.**
- **MAE (mean absolute error)**
  - 22.14 Bikes
  - This is significantly higher than what we were looking for
- **These results allow us to conclude that prophet was not an optimal model for predicting occupancy or the total number of bikes at a given station on our dataset**
  - Our dataset had very weak and varied seasonal patterns as well as substantial noise/variance
  - Prophet is essentially trying to formulate a curve (taking information regarding our series into account when forming to essentially produce a transformation that can map our data to-and-from a linear or log function) to fit our dataset. Despite the additional components that prophet accounts for, it still suffers from some of the same problems of a traditional linear or log regression.
  - Our data is temporally dependent, so a model that does not perform regressions and accounts for the fact that our data forms a time-series would be most effective
    - We have shown this in the results previously presented for ARIMA and LSTM
  - As an aside, prophet has also been quite slow to train, as it does not support any form of GPU or TPU acceleration.

Comparison & Conclusion

- XGBoost achieves the strongest performance across all targets, with the highest R2 and lowest RMSE.
- LSTM captures smooth temporal trends but exhibits higher variance during abrupt congestion events.

Table 1: Available Bikes Forecasting Performance

Model	RMSE	MAE	R^2	MAPE (%)
XGBoost	**2.09**	**1.18**	**0.957**	23.19
LSTM	3.75	2.10	0.862	39.13
ARIMA	2.62	2.19	0.012	25.53
ARIMAX	2.57	2.08	0.046	24.58

Table 2: Available Docks Forecasting Performance

Model	RMSE	MAE	R^2	MAPE (%)
XGBoost	**3.09**	**1.81**	**0.940**	22.46
LSTM	4.31	2.49	0.884	31.90
ARIMA	2.52	1.96	0.018	11.87
ARIMAX	2.68	2.13	-0.116	12.22

Table 3: Occupancy Forecasting Performance

Model	RMSE	MAE	R^2	MAPE (%)
XGBoost	**0.061**	**0.037**	**0.959**	22.76
LSTM	0.111	0.065	0.867	38.94
ARIMA	0.066	0.052	0.101	15.94
ARIMAX	0.066	0.052	0.115	15.89

Streamlit Application

- This Streamlit application operationalizes our forecasting models into a real-time congestion intelligence system.
  We ingest live Citi Bike GBFS feeds and visualize station conditions on an interactive map with region-based filtering and congestion risk labels.
- Users can select any station to compare actual versus predicted occupancy using XGBoost, LSTM, and ARIMA, with full diagnostic plots for model evaluation.
- Overall, the application demonstrates how our machine learning models can support proactive bike rebalancing and real-world operational decision-making.

Future Work

While the current project demonstrates strong performance in short-term bike availability forecasting using machine learning models such as XGBoost, ARIMA, and LSTM, several promising extensions can further enhance its operational impact, scalability, and intelligence.

- **Station Congestion Classification:**
  Extend forecasting to probabilistic classification to predict full/empty station risk within 30–60 minutes and trigger proactive congestion alerts.

- **Dynamic Bike Rebalancing Optimization:**
  Integrate demand forecasts with optimization techniques (MILP, heuristics) to recommend optimal bike rebalancing schedules and truck routing strategies.

- **Reinforcement Learning for Autonomous Rebalancing:**
  Apply RL methods (DQN, PPO) to learn adaptive, real-time bike redistribution policies under dynamic demand and traffic conditions.

- **Anomaly and Event Detection:**
  Incorporate anomaly detection to identify sudden demand spikes caused by events, weather disruptions, or transit outages.

- **Critical Station & Bottleneck Identification:**
  Use network analytics and SHAP-based explainability to identify high-impact stations and understand the drivers of system instability.

- **Real-Time and City-Scale Deployment:**
  Enable real-time inference using live data streams and extend the framework to support scalable, multi-city bike-sharing systems.

References

- *[1] Taylor, S. J., and Letham, B. (2018). Forecasting at scale. The American Statistician, 72(1):37–45.*
- *[2] Yao, H., Tang, X., Wei, H., Zheng, G., & Li, Z. (2018). Modeling spatial-temporal dynamics for traffic prediction using graph convolutional networks. AAAI Conference on Artificial Intelligence*
- *[3] Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control. Wiley.*
- *[4] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD.*
- *[5] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.*
- *[6] Taylor, S. J., & Letham, B. (2018). Forecasting at Scale. The American Statistician.*
- *[7] Yao, H., et al. (2018). Modeling Spatio-Temporal Dynamics Using Graph Convolutional Networks. AAAI.*
- *[8] Hyndman, R. J., & Athanasopoulos, G. (2018). Forecasting: Principles and Practice. OTexts.*

STEVENS
INSTITUTE OF TECHNOLOGY
1870

This project moves Citi Bike analytics from descriptive to predictive and actionable intelligence.

THANK **YOU**

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030

Questions?